Operating Systems and Parallel Computing

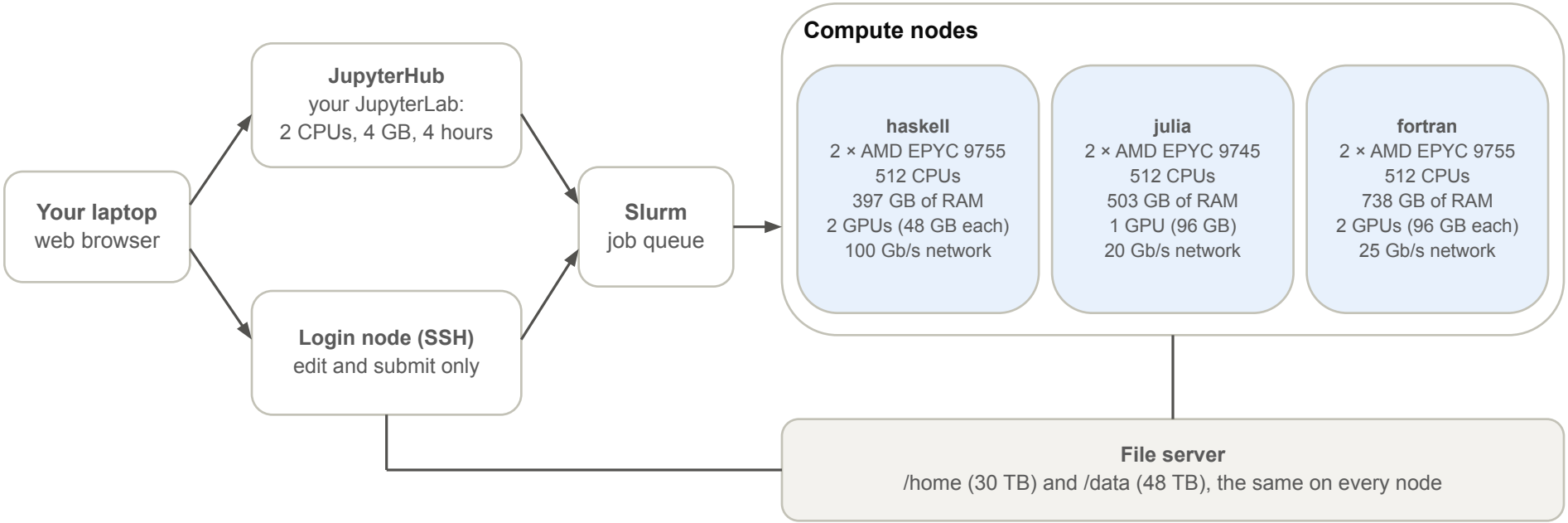
Lecture 14

Prof. Matteo Turilli

Quiz 1

- **30 minutes**, 10 questions on Lab 1, on paper.
- Laptops, phones, and smartwatches closed and put away until the end of the quiz.
- Write your FULL NAME in capitals, and **follow the instructions on the answer sheet**.
- When you finish, please deliver the sheet to me and leave. **Come back to class at the end of the 30 minutes**.
- After the quiz: your first login and your first batch job on the cluster; then threads, the GIL and processes.

Our Cluster



- JupyterHub and the login node hand your jobs to Slurm, which runs them on the compute nodes.
- Your files in `/home` are the same everywhere.

Each compute node also has a local disk, `/scratch`, emptied after 14 days.

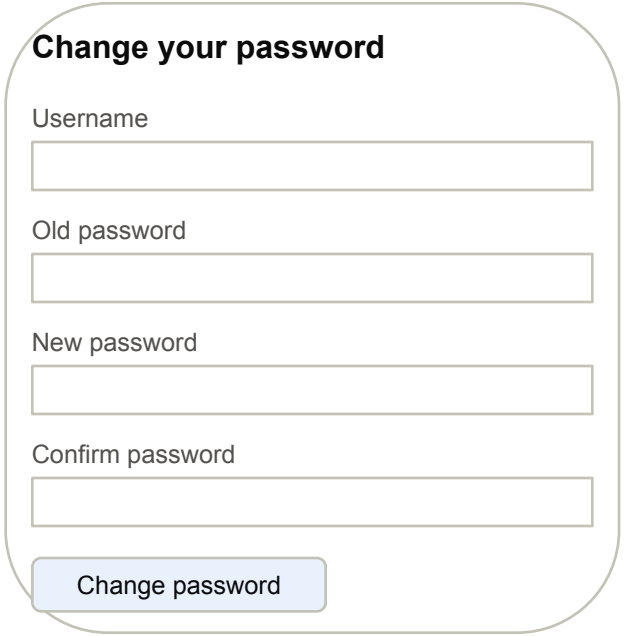
Develop in JupyterHub, Measure in Batch Jobs

- **JupyterHub** (`jupyter.iesci.tech:8085`): your JupyterLab runs as a small job on a compute node, with 2 CPUs and 4 GB for up to 4 hours. Write and test code on small inputs, and submit jobs from its terminal.
- **Batch jobs**: measurements and full-size runs, with the CPUs, memory and time you ask for. All the measurements of your group project run as batch jobs.
- **Never run code on the login node**. With SSH you land on a small machine shared by everyone; NumPy does not even start there.
- **JupyterLab stops on its own**: a notebook idle for 30 minutes loses its variables, and the session ends after 60 idle minutes.
- **Your account is temporary**: on 31 December logins stop and running jobs are cancelled. Keep code and results in your team's GitHub repository.

This is an interactive class

1. Take your paper slip: your **username** (pc01 to pc94) and your **temporary password**.
2. JupyterHub does not accept a temporary password: change it first. Open the password page, `https://password.iesci.tech:8085` .
3. **Username**: yours. **Old password**: the temporary one. **New password** and **Confirm password**: at least 10 characters, with a lowercase letter, an uppercase letter and a digit. Click **Change password**.
4. Wait 30 seconds. Open `https://jupyter.iesci.tech:8085` and sign in with your new password. No VPN is needed.

Forgot your new password? Email `support@iesci.tech` . The JupyterHub sign-in page also links to the password page.



https://password.iesci.tech:8085

JupyterLab and the Course Kernel

- Wait for JupyterLab to start: your session is a small job on a compute node, with 2 CPUs and 4 GB, for up to 4 hours.
- Register the course kernel, once. Open a terminal (File > New > Terminal) and run:

```
/data/software/envs/ospc/bin/python -m ipykernel install --user \
    --name ospc --display-name "OSPC (Python 3.12)"
```

- Reload the page: **OSPC (Python 3.12)** appears in the Launcher. Use it for your notebooks.
- Check in: in the terminal, run `whoami` and paste what it prints into the Blackboard test **Cluster check-in** (Assignments, Project, Exams) before the end of class. **No check-in: 3 points off your final score** (an 8 becomes a 5).
- Keep the terminal open: Exercise 1 starts there.
- Every step, and what to do when something fails, is in the cluster guide (Blackboard, Supporting Material).

Slurm in One Slide

- **Slurm** keeps the queue: you say what your job needs, and Slurm starts it when those resources are free (space sharing, Lecture 12).
- A **partition** is a queue with its own rules:

Partition	For	Time limit
`cpu`	batch jobs (the default)	7 days
`gpu`	jobs that need a GPU	3 days
`interactive`	JupyterLab sessions	8 hours

- **Student limits**: 64 CPUs and 1 GPU in use at once; 4 running and 8 submitted jobs. The whole class shares 256 CPUs and 2 GPUs: a job can wait while CPUs are free.
- A job sees only its own CPUs: `nproc` inside a job prints what Slurm gave it.
- Two jobs may share the same cores, so timings vary: repeat each measurement and report the spread.
- Interactive version: mturilli.github.io/ospc-materials/animations/slurm-queue.html

Exercise 1: Your First Batch Job

- Type the commands in the terminal of your JupyterLab.
- **Predict**: how many CPUs does your JupyterLab session have?

  ```
  nproc
  ```
- **Predict**: this job asks for 4 CPUs and runs a Python loop. What will `nproc` print inside the job? How much of its 4 CPUs will it use?

  ```
  export PATH=/data/software/envs/ospc/bin:$PATH
  W='sum(i*i for i in range(300_000_000))'
  sbatch -p cpu -c 4 --mem=2G -t 5 --wrap "nproc; python -c '$W'"
  squeue -u $USER
  ```
- `--wrap` turns a command into a job script. `-c 4` means `--cpus-per-task=4`; `-t 5` means 5 minutes.

Exercise 1: What Did the Job Use?

- **TotalCPU**: the CPU time of all the job's processes. **Efficiency** = TotalCPU / (Elapsed × AllocCPUS).
- **Predict**: your job ran one Python loop with 4 CPUs. What will its efficiency be?
- When the job has left the queue (about 10 s), read its output and its accounting. Use your job ID:

  ```
  cat slurm-ID.out
  ```

  ```
  sacct -j ID -o JobID,Elapsed,TotalCPU,AllocCPUS,State
  ```
- Elapsed is in whole seconds, so for a job of a few seconds the efficiency is a rough number.

Exercise 1: Questions

1. What did `nproc` print in your JupyterLab session, and inside the job? Why?
2. What were Elapsed, TotalCPU and AllocCPUS of your job? What was its efficiency?
3. Your team's script runs for 10 minutes on 32 CPUs, and `sacct` shows a TotalCPU of 20 minutes. What do you conclude? What do you change?

Exercise 1: Answers

1. What did `nproc` print in your JupyterLab session, and inside the job? Why?
2. What were Elapsed, TotalCPU and AllocCPUS of your job? What was its efficiency?
3. Your team's script runs for 10 minutes on 32 CPUs, and `sacct` shows a TotalCPU of 20 minutes. What do you conclude? What do you change?

1. 2 in the session and 4 in the job. A job sees only the CPUs that Slurm gave it, and JupyterHub asks for 2.
2. On our cluster: 10 s elapsed, 9.3 s of CPU time, 4 CPUs: 9.3 / (10 × 4) ≈ 23%. One CPU worked and three waited: a Python loop runs on one core, however many CPUs the job has.
3. 20 / (10 × 32) ≈ 6%: the script kept about 2 of its 32 CPUs busy. Ask for 2 CPUs, so the job starts sooner and leaves cores to others, or make the code parallel: the rest of today.